



Indian Institute of Technology Indore



Science and Technology Council

IITI-SOC 2026

End Evaluation Report

For

QUANTUM RL AGENT FOR F1 RACE
STRATEGY OPTIMIZATION

Team: QC-3

Team Members:

1. Mayank Gupta – 250001047 (Leader)
2. Mayank Jha – 250004023
3. Chetna Mundra – 250001024
4. Piyushi – 250001055

Submission Date: 30-07-2026

GitHub Repository:

<https://github.com/the-Monk-bro/F1-strategy-optimization>

Contents

1	Problem Statement	2
2	The Race Simulation Environment	2
2.1	Real Race Data	2
2.2	What the Agent Can See (Observations)	2
2.3	What the Agent Can Do (Actions)	3
2.4	Simulated Lap Time Calculation	3
2.5	Race Session (Standings Tracking)	3
2.6	Rewards – How the Agent Knows If It Did Well	4
2.7	Action Masking	4
3	Classical DQN Agent	4
3.1	The Neural Network	4
3.2	Key DQN Improvements Used	5
3.3	DQN Hyperparameters	5
3.4	The Training Loop	5
3.5	Checkpoint Saving	6
3.6	DQN Training & Performance Plots	6
4	Quantum RL Agent	8
4.1	Quantum-classical hybrid neural network	8
4.2	How the Quantum Circuit Works	8
4.3	Avoiding Barren Plateaus	9
4.4	QRL vs DQN Differences	9
4.5	QRL Training & Performance Plots	9
5	Benchmarking and Visualization	12
5.1	Baseline Agents and Real Driver Strategy	12
5.2	Benchmarking Pipeline	12
5.3	Streamlit Interactive Dashboard	12
6	Justification of Final Approach	13
6.1	Quantum circuit architectures	13
6.2	Why v9, v10, v11 didn't work?	13
6.3	Result Comparisons	13
6.4	DQN vs QRL Comparative Curves	14
7	Project Architecture	16
8	Conclusion and Future Work	16

1 Problem Statement

Formula 1 pit stop strategy is a sequential decision-making problem under uncertainty: at every lap, a race strategist must decide whether to pit, and if so, which tyre compound to fit, while weighing tyre degradation, track position, rival strategies, and the time cost of a stop.

Real F1 teams use complex computer models and experienced engineers to make these calls. This project attempts to train an AI agent to learn good pit stop strategy automatically, just by racing in a simulated environment.

This project builds a system that can make smart pit stop decisions in a Formula 1 race. This project frames the problem as a reinforcement learning (RL) task and our ultimate goal is to compare the performance of two types of learning agents:

- **A classical RL agent** — uses a normal neural network (Deep Q-Network or DQN)
- **A quantum RL agent** — uses a variational quantum circuit combined with the same pipeline as the classical agent.

2 The Race Simulation Environment

This is the heart of the project. The environment simulates a real F1 race, lap by lap.

2.1 Real Race Data

The environment uses **real historical F1 race data** from three circuits and three years:

1. **Monaco** (2022, 2023, 2024)
2. **Monza** (2022, 2023, 2024)
3. **Silverstone** (2022, 2023, 2024)

This gives 9 different races. When training starts, a random race is picked from these 9 options. This ensures the agent learns general strategy, not just one specific track's patterns.

2.2 What the Agent Can See (Observations)

Each lap, the agent receives **15 numbers** describing the current race situation:

#	What it Tells the Agent
0	How far into the race (e.g., 0.5 = halfway)
1	Current position in the race (normalized)
2	How fast the car is compared to the fastest car
3–7	Which tyre is currently on the car (one-hot encoding for 5 tyre types)
8	How worn out the current tyres are
9	How far behind the race leader
10	Gap to the car directly ahead
11	Gap to the car directly behind
12	Safety car status (0 = green flag, 0.5 = VSC, 1 = full SC)
13	Track wetness (0 = dry, 0.5 = damp, 1 = wet)
14	Whether the 2-dry-compound rule has been satisfied

All values are normalized between 0 and 1, which helps the neural network learn more efficiently.

2.3 What the Agent Can Do (Actions)

The agent has **6 possible actions** each lap:

Action	Meaning
0	Stay out — keep racing without pitting
1	Pit and fit Soft tyres
2	Pit and fit Medium tyres
3	Pit and fit Hard tyres
4	Pit and fit Intermediate tyres
5	Pit and fit Wet tyres

2.4 Simulated Lap Time Calculation

The key formula for calculating how long each lap takes is:

$$\text{Lap Time} = \text{Base Track Time} + \text{Tyre Degradation} + \text{Pit Loss} + \text{Traffic Loss} + \text{Weather Penalty} + \text{Random Noise}$$

Base Track Time: Each tyre compound has a different base speed on the track.

Tyre Degradation: As tyres get older, the car gets slower. The model uses pre-computed degradation curves from real data. A soft tyre degrades faster than a hard tyre.

Pit Loss: A pit stop costs around 20-25 seconds. Under safety car conditions, this is halved (because all cars are already slowing down).

Traffic Loss: If the car ahead is very close (less than 1 second gap), the driver loses time because they can't overtake. This penalty is 0.5 seconds per lap when very close, 0.4 seconds when close, and 0 when there is clear air.

Weather Penalty: Using the wrong tyre for conditions causes huge time losses:

4. Wet tyres on a dry track: +15 seconds per lap
5. Dry tyres on a fully wet track: +40 seconds per lap

Safety Car: If a Full Safety Car is active, all lap times are multiplied by 1.35 (35% slower). For a Virtual Safety Car, they are multiplied by 1.18 (18% slower).

Random Noise: A small random variation is added to each lap time to make the simulation realistic (real races are never perfectly predictable).

2.5 Race Session (Standings Tracking)

The RaceSession module keeps track of all 20 drivers on track in parallel. After each lap:

6. The agent's simulated time is recorded.
7. All other drivers' real historical lap times are used.
8. All drivers are sorted by total race time.
9. Gaps between cars are recalculated.

This is a clever design: the agent competes against real historical drivers, making the simulation realistic.

2.6 Rewards – How the Agent Knows If It Did Well

The reward signal tells the agent whether each decision was good or bad. The reward system has several components:

Per-lap reward:

$$\text{Reward} = (\text{Positions gained this lap} \times 1.0) + (\text{Gap closed to leader} \times 0.1) - (\text{Pace deficit} \times 0.1)$$

Pit stop penalty: Every pit stop costs -5 reward points (because pit stops temporarily lose time and positions).

Early pit stop penalty: If the agent pits with tyres that are less than 5 laps old, an extra -20 penalty applies (because pitting too early wastes good tyres).

Rule violation penalty: If the race ends and the agent didn't use at least 2 different dry tyre compounds, it gets -150 points (a very large penalty to teach the agent the rule).

Championship points bonus: At the end of the race, the agent earns F1 championship points based on its finishing position (25 for 1st, 18 for 2nd, ..., 1 for 10th). This is the main objective — the agent learns to finish higher up the order.

2.7 Action Masking

Rather than letting the agent freely pick any action and learn from bad mistakes, we implement **action masking** — a system that blocks clearly illegal or nonsensical actions before the agent even sees them:

1. No pitting in the first 2 laps (race has just started)
2. No pitting again if the tyre is only 1-2 laps old (would waste fresh tyres)
3. No pitting on the final lap (too late to matter)
4. No dry tyres when it's a wet track
5. No intermediate or wet tyres when it's dry
6. No pitting onto the same compound already on the car

Action masking speeds up learning significantly because the agent never wastes time exploring obviously bad choices.

3 Classical DQN Agent

The classical agent uses a fully classical multilayer network with dueling value/advantage heads to approximate $Q(s,a)$ from the 15-dimensional state. It serves as the control arm of the experiment.

3.1 The Neural Network

The agent uses a Dueling DQN network architecture. Instead of one output, there are two branches:

1. Value stream: “How good is my current situation overall, regardless of which action I take?”
2. Advantage stream: “How much better or worse is each specific action compared to an average action?”

These are combined as: $Q(s, a) = \text{Value}(s) + \text{Advantage}(s, a) - \text{mean}(\text{Advantage}(s, \cdot))$

This design helps the agent learn more stably, especially in situations where most actions give roughly the same result (common in racing where in most laps, staying out is fine).

The network has 2 hidden layers of 256 neurons each, with Layer Normalization for stable training.

3.2 Key DQN Improvements Used

Double DQN: Prevents the agent from overestimating Q-values. The online network selects which action to take next, but the target network estimates how good that action is. This separation reduces a known bias in standard DQN.

Prioritized Experience Replay (PER): Instead of randomly sampling past experiences to learn from, the agent prioritizes experiences where it made bigger mistakes (higher TD error). This means the agent focuses its learning on the situations where it has the most to improve.

Soft Target Network Update: There are two networks - the online network (updated every step) and the target network (updated slowly). The target network is used to calculate what the “ideal” Q-value should be. This prevents the training target from shifting too rapidly. Update formula: $\text{target} = 0.005 \times \text{online} + 0.995 \times \text{target}$

Epsilon-Greedy Exploration: The agent starts by picking random actions ($\epsilon = 1.0 = 100\%$ random) and gradually shifts to using its learned strategy (ϵ decays to $0.05 = 5\%$ random) over 80,000 steps. This ensures the agent explores broadly early on before committing to learned patterns.

3.3 DQN Hyperparameters

Parameter	Value	Why
Learning rate	0.0001	Small enough for stable learning
Batch size	128	Balance between speed and stability
Replay buffer size	100,000	Stores recent experiences
Discount factor (γ)	0.99	Agent cares a lot about future rewards
Gradient clip	10.0	Prevents learning instability
Epsilon decay	80,000 steps	Gives plenty of time to explore

3.4 The Training Loop

Each **episode** is one full simulated race from start to finish. Here’s what happens:

Start of episode:

- Pick a random race from 9 available (Monaco/Monza/Silverstone $\times$ 2022/2023/2024)
- Pick a random eligible driver as the agent

Each lap:

- Agent looks at 15 race state numbers
- Apply action mask (block illegal actions)
- Agent picks an action (pit or stay out)
- Simulation calculates new lap time
- Race standings are updated
- Agent receives a reward

- Experience is stored in replay buffer
- Training step: sample from buffer, update network weights

End of race:

- Record final position, total reward, pit count

The DQN agent trains for **1,500 episodes**. Each episode spans 50–78 laps depending on the circuit. That’s roughly 75,000–117,000 decision points total.

3.5 Checkpoint Saving

The training system saves three model files:

1. `latest.pt` — saved every 50 episodes (for recovery if training crashes)
2. `best.pt` — saved whenever the rolling-50-episode average reward reaches a new high
3. `final.pt` — saved after all 1,500 episodes are complete

This lets us compare the “best ever” performance against the “final” performance, which is useful for detecting overfitting or instability late in training.

3.6 DQN Training & Performance Plots

Below are the empirical training curves and pit stop distribution for the classical DQN agent across 1,500 training episodes:

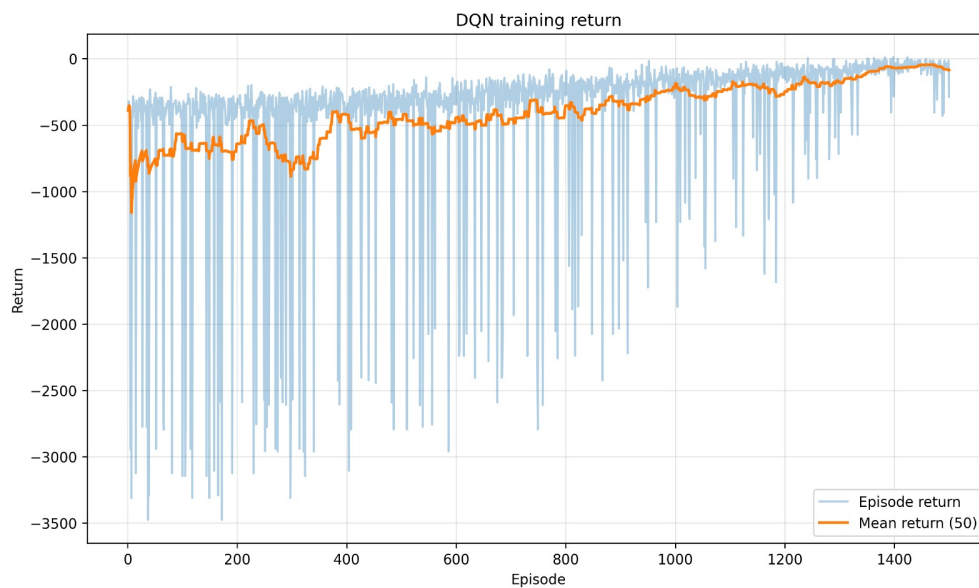


Figure 1: *Classical DQN training return per episode (rolling 50-episode average).*

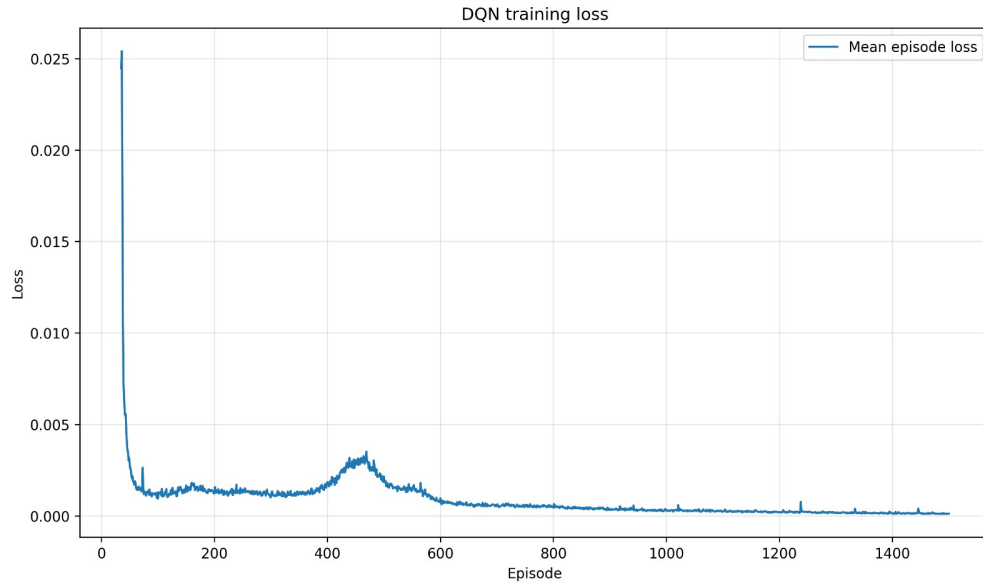


Figure 2: *Classical DQN temporal difference (TD) loss curve across training steps.*

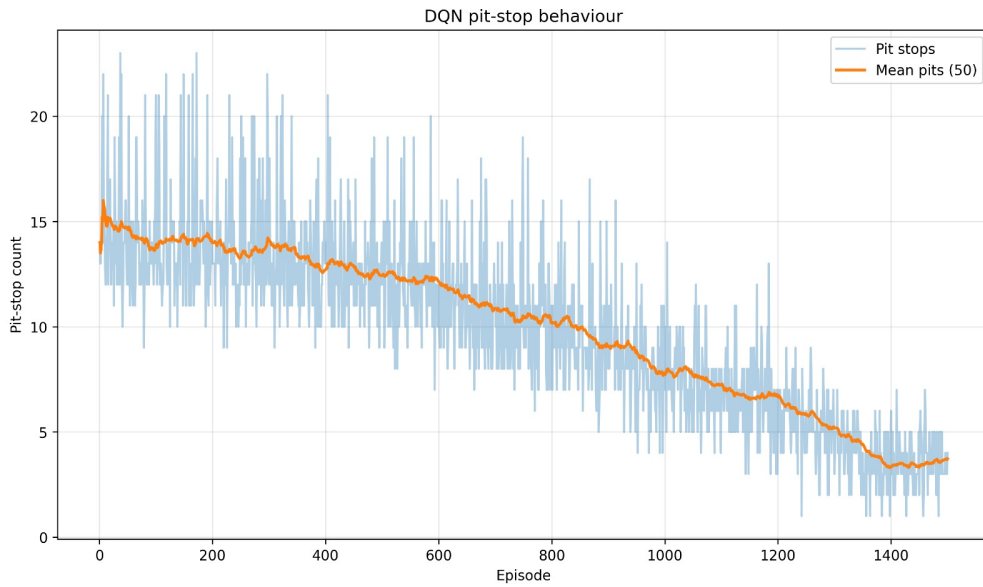


Figure 3: *Classical DQN pit stop frequency (rolling 50-episode average).*

4 Quantum RL Agent

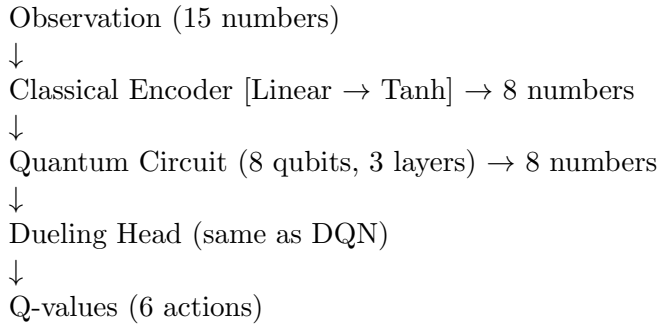
We build a quantum-classical hybrid neural network that plugs into the *exact same* DQN training loop we have already built for the classical agent. The network takes the race state, squeezes it through a small quantum circuit sandwiched between two classical layers, and outputs Q-values. We then train it with experience replay and a target network (identical machinery to classical DQN), so that any performance difference we will measure will come from the quantum layer itself, not from different training conditions.

4.1 Quantum-classical hybrid neural network

The key features of this QRL agent are:

- State features are encoded into an 8-qubit variational quantum circuit via angle embedding (`qml.AngleEmbedding`).
- 3 variational layers of single-qubit rotations (`qml.Rot`) interleaved with ring-topology CNOT entangling gates.
- Expectation values of Pauli-Z observables are read out per qubit (`qml.expval`) and fed into the same dueling value/advantage head architecture used downstream by the classical agent, keeping the post-quantum processing directly comparable.
- Implemented as a `qml.qnn.TorchLayer` so the circuit trains end-to-end with standard PyTorch backpropagation inside the same Double DQN + PER loop as the classical agent.
- Simulated on PennyLane’s default.qubit simulator (see Section 6 for the simulation-vs-hardware scope decision).

The quantum architecture looks like:



4.2 How the Quantum Circuit Works

Step 1 - Angle Embedding:

Each of the 8 numbers from the encoder is used as a rotation angle on one qubit. This encodes the current race state into the quantum system.

Step 2 - Variational Layers (3 layers):

Each layer applies rotation gates (Rot gates = 3 parameters per qubit) followed by ring entanglement. “Ring entanglement” means each qubit is connected to the next one in a circle using CNOT gates:

$$\text{Qubit } 0 \rightarrow \text{Qubit } 1 \rightarrow \text{Qubit } 2 \rightarrow \dots \rightarrow \text{Qubit } 7 \rightarrow \text{Qubit } 0$$

Entanglement allows information to flow between qubits, enabling the circuit to learn complex patterns.

Step 3 - Measurement:

The expectation value of the Pauli-Z operator is measured on each qubit. This gives 8 real numbers as output, which feed into the dueling heads.

Total quantum parameters: 3 layers $\times$ 8 qubits $\times$ 3 rotation angles = **72 trainable quantum parameters**

4.3 Avoiding Barren Plateaus

A known problem with quantum circuits is the **barren plateau** problem — if weights start too large, gradients become exponentially small and training fails. This project fixes it by initializing quantum weights with very small random values (standard deviation = 0.01), keeping the circuit close to the identity operation at the start where gradients are larger.

4.4 QRL vs DQN Differences

Property	DQN	QRL
Network body	Classical MLP	Quantum circuit
Hidden dimensions	256 neurons	8 qubits
Learning rate	0.0001	0.005 (higher)
Batch size	128	32 (smaller because quantum is slower)
Training speed	Fast	Slower (quantum simulation is expensive)
Device	GPU supported	CPU only (quantum simulation)

The dueling heads and all other training logic (PER, Double DQN, soft updates) are **identical** to the DQN agent. This is intentional — any performance difference observed can be directly attributed to the network body (classical vs. quantum), not to other factors.

4.5 QRL Training & Performance Plots

Below are the empirical training curves and pit stop distribution for the hybrid Quantum RL (v6) agent across 1,500 training episodes:

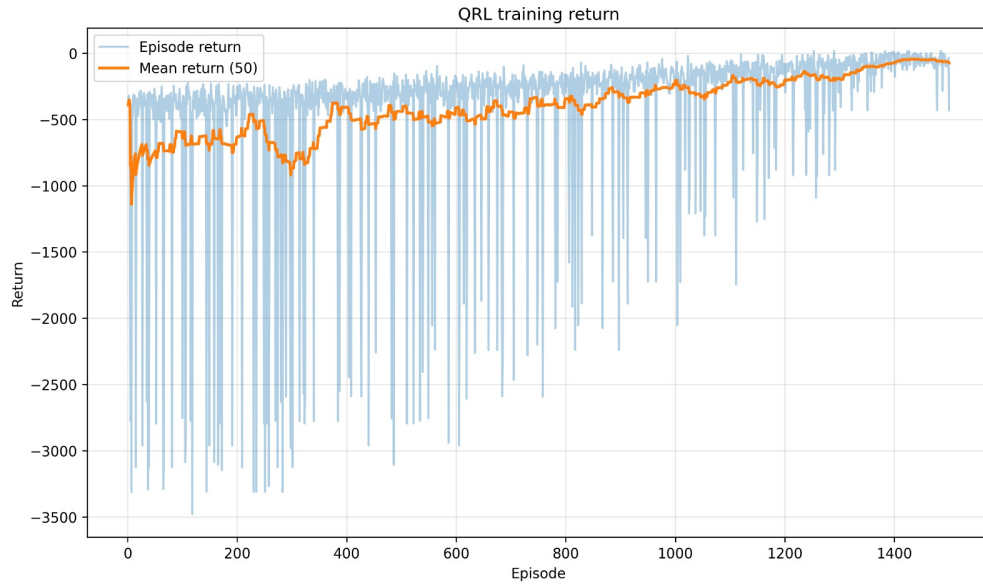


Figure 4: *Hybrid QRL training return per episode (rolling 50-episode average).*

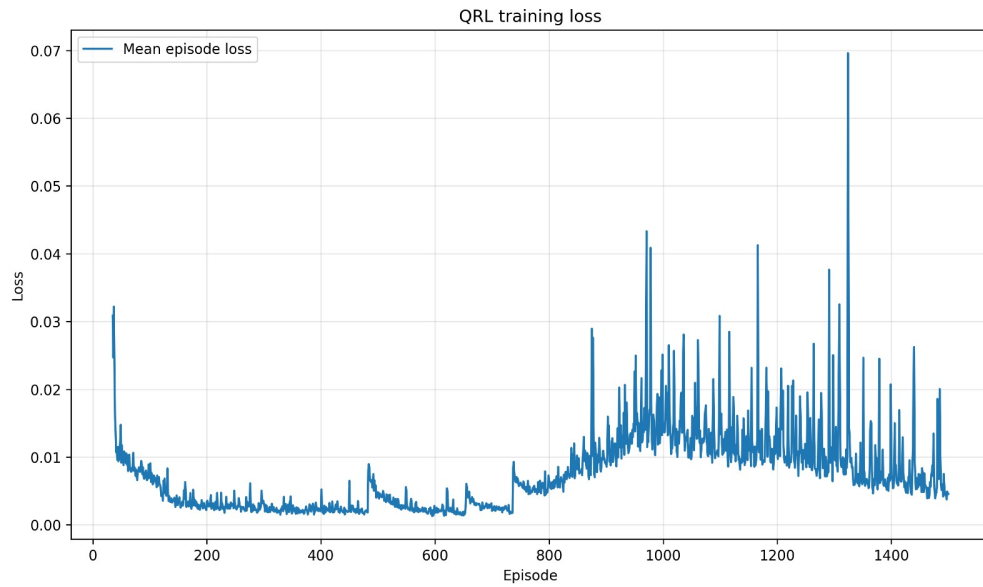


Figure 5: *Hybrid QRL temporal difference (TD) loss curve across training steps.*

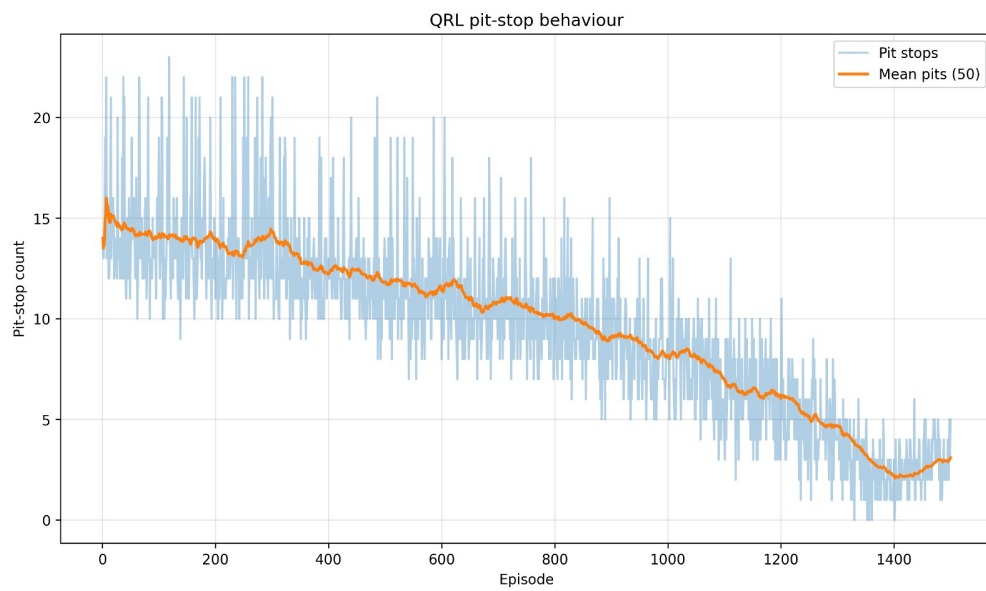


Figure 6: *Hybrid QRL pit stop frequency (rolling 50-episode average).*

5 Benchmarking and Visualization

To evaluate and analyze the performance of our trained agents, we built an automated benchmarking pipeline and an interactive visualization dashboard. This provides a structured framework to compare the quantum RL agent against the classical DQN agent, deterministic baseline policies, and real-world F1 driver strategies.

5.1 Baseline Agents and Real Driver Strategy

To evaluate learning progress against meaningful reference points, our benchmarking suite implements four reference strategies alongside the DQN and QRL agents:

- **Random Agent:** Picks randomly among legal actions allowed by the action mask to establish a zero-intelligence lower bound.
- **Always Stay Out Agent:** Never pits (executes action 0 every lap) to evaluate race time and tyre degradation without any pit stop time loss.
- **Rule-Aware Heuristic Agent:** A deterministic rule-based strategy that pits when tyre age exceeds thresholds (e.g., 18+ laps) or during track wetness transitions.
- **Real Driver Strategy:** Replays the actual historical pit stop decisions made by real F1 drivers in fastf1 race data as a human benchmark.

5.2 Benchmarking Pipeline

The benchmarking system (in `benchmarking/`) runs systematic evaluation episodes across standard tracks, seasons, and random seeds. Key capabilities include:

- **Automated Metric Aggregation:** Evaluates performance across key race metrics including Total Race Reward, Final Position, Position Gain, Pit Stop Count, and Mean Lap Time.
- **Artifact Generation:** Outputs standardized CSV report files (`overall_metrics.csv`, `track_metrics.csv`, `paired_comparison.csv`, and `lap_trace.csv`) saved under `benchmarking/artifacts/latest/`.
- **Statistical Significance Testing:** Computes 95% confidence intervals and paired statistical comparisons (QRL vs. DQN) across matching race seeds.

5.3 Streamlit Interactive Dashboard

We built an interactive Streamlit web application (in `dashboard/`) to analyze benchmark artifacts through three distinct visualization pages:

- **Overview Page (`1_Overview.py`):** Displays summary metric bar charts with 95% confidence intervals, overall win-rate breakdowns, and paired statistical agent comparisons.
- **Lap Replay Page (`2_Lap_Replay.py`):** Provides a step-by-step lap replay of selected races, visualizing position changes, gap to leader, tyre degradation, and pit stop calls over time.
- **Strategy Divergence Page (`3_Strategy_Divergence.py`):** Highlights specific laps where the QRL agent and DQN agent chose different strategic actions, allowing detailed analysis of strategic decision differences.

6 Justification of Final Approach

Since this project was about comparing and optimizing the performance of the quantum RL agent in comparison to the DQN agent, multiple approaches were tried to optimize the QRL agent.

6.1 Quantum circuit architectures

While working on the quantum RL agent, multiple versions were tested altering the quantum circuit architecture in each one to see which one performed the best before choosing the final version for benchmarking.

v6 was the best-performing checkpoint overall which is the one we used with angle embedding, 3 variational layers and ring entanglement topology.

6.2 Why v9, v10, v11 didn't work?

After v6, further quantum-architecture experiments were tried on top of it. None of them cleanly beat v6 — each produced trade-offs (some things improved, others got worse) rather than a clean win, and checkpoint stability got worse compared to v6.

What the architecture changes actually were

- **v9 (Wider head):** widened the classical layers sitting after the quantum circuit, to test whether the bottleneck was downstream capacity rather than the quantum circuit itself. Since the result didn't improve, it suggests that the limiting factor isn't how much classical processing happens *after* the quantum circuit, but the information content coming *out* of the 8-qubit circuit itself.
- **v10 (Pairwise correlator readout):** changed how the circuit's output was read out — using pairwise correlations between qubits instead of individual per-qubit measurements to test whether richer correlation information helped. The reasoning for its failure is that this state representation may simply not carry extra useful information in higher-order qubit correlations for this particular task.
- **v11 (Data re-uploading):** re-encoded the input state at multiple points inside the circuit (instead of once at the start), to test whether this increased the circuit's expressive power. Re-uploading increases effective circuit depth, and greater depth is exactly the direction that makes barren plateaus worse. Therefore, it is possible the added expressivity came at the cost of trainability.
- **v12 (Reduced circuit depth):** Tested the opposite idea - a shallower circuit, on the theory it would train more reliably but still the results didn't improve probably because the depth was cut back too far, it likely gave up expressive power the circuit needed, trading stability for capacity rather than finding a better balance than v6.

6.3 Result Comparisons

Below are the result comparisons for each of the QRL agent checkpoints:

Checkpoint_qrl	Return	Position	Pits	Worst pos
v6 (final)	-33.44	3.85	1.95	12
v9 (wider head)	-43.34	4.5	1.3	12
v10 (correlators)	-27.34	4.05	1.80	12
v11 (reuploading)	-33.30	4.45	2.15	12

Checkpoint_qrl	Return	Position	Pits	Worst pos
v12 (2 layers)	-27.76	4.30	1.65	13

6.4 DQN vs QRL Comparative Curves

Below are the direct head-to-head performance comparison curves comparing the Classical DQN agent against the Hybrid QRL agent across all benchmarking episodes:

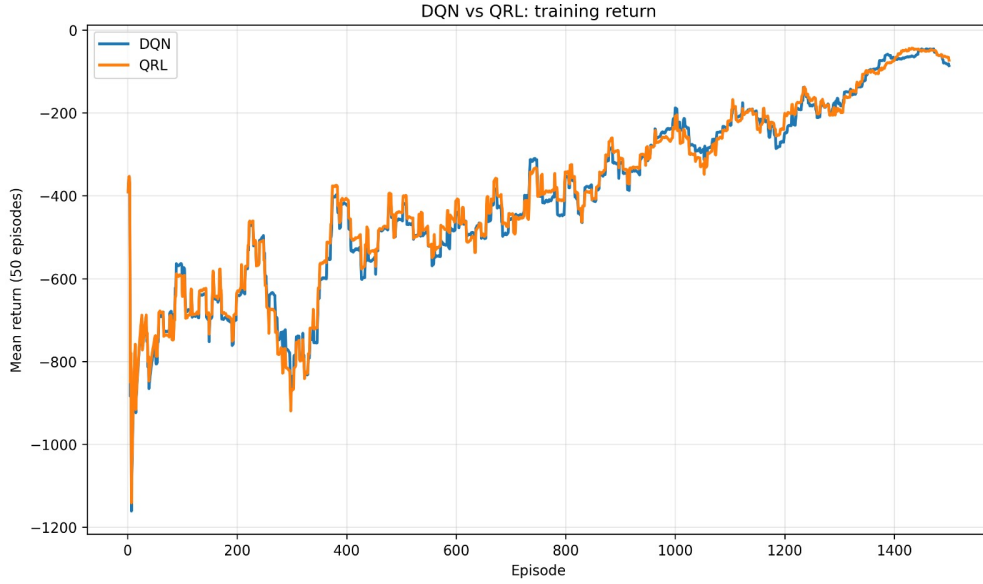


Figure 7: *Comparative training return curves (DQN vs QRL) over 1,500 training episodes.*

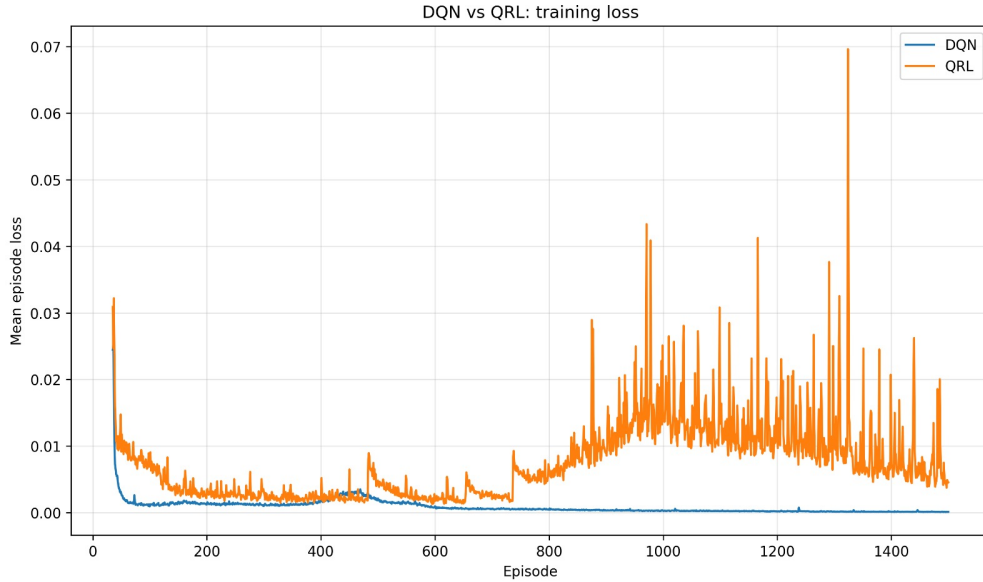


Figure 8: *Comparative training loss curves (DQN vs QRL) across training steps.*

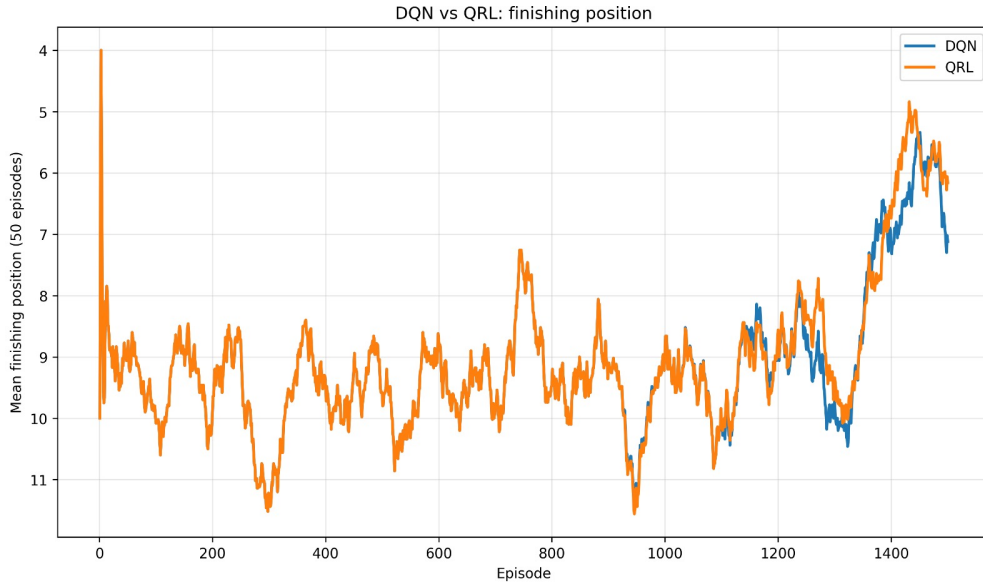


Figure 9: *Comparative race finishing position distribution between Classical DQN and Hybrid QRL.*

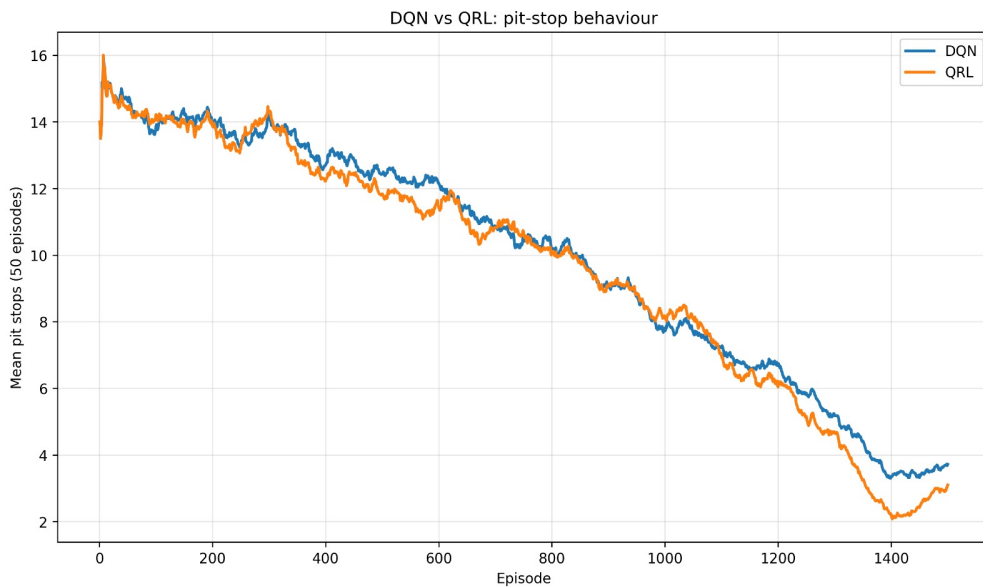


Figure 10: *Comparative pit stop frequency distribution between Classical DQN and Hybrid QRL.*

7 Project Architecture

The project is cleanly structured into the following main components:

```
project-root/
|
|-- env/                <- Race simulation (track models, tyre degradation,
|                        session standings)
|-- agents/             <- Reinforcement learning agents & baseline strategies
|   |-- dqn/            <- Classical Deep Q-Network agent (dueling architecture, PER)
|   |-- qrl/            <- Hybrid Quantum RL agent (PennyLane 8-qubit circuit)
|   |-- baselines.py    <- Standard benchmark baseline policies (Random,
|   |                    Stay-Out, Heuristic)
|   '-- real_agent.py   <- Real driver historical strategy replay policy
|-- training/           <- Training scripts and loops for DQN and QRL agents
|-- benchmarking/       <- Automated evaluation pipeline, metric aggregation,
|                        and CSV artifacts
|-- dashboard/          <- Interactive Streamlit web application for visual
|                        analysis & replays
|-- checkpoints/        <- Saved model files (.pt) after training
|-- evaluation/         <- Benchmark evaluation runner scripts
|-- data/               <- Historical F1 race datasets (FastF1 data)
'-- reports/            <- Evaluation reports and documentation
```

Each component connects smoothly with the rest: the environment simulates the race, the agents make strategy decisions, training updates the models, benchmarking evaluates performance across seeds, and the dashboard visualizes performance and lap replays.

8 Conclusion and Future Work

Working on this project, we were able to successfully achieve:

1. **A complete F1 race simulation** using real historical data from 9 races across 3 tracks and 3 years, with realistic tyre degradation, pit stop modeling, traffic effects, weather handling, and safety car logic.
2. **A classical DQN agent** implementing state-of-the-art techniques: Dueling architecture, Double DQN, Prioritized Experience Replay, action masking, and soft target updates.
3. **A quantum reinforcement learning agent** using an 8-qubit variational quantum circuit as the network body, with the same training framework as the DQN for a fair comparison.
4. **A complete dashboard** for visualizing the benchmarking results.

What Could Be Improved:

1. **More races and tracks:** Expanding beyond 9 races would improve generalization.
2. **Multi-agent training:** Training the agent alongside other learning agents (not just historical drivers) could lead to better emergent strategies.
3. **Real quantum hardware:** Running the QRL agent on actual quantum hardware (not simulation) could reveal true quantum advantages.
4. **More complex observations:** Including tyre temperature, fuel load, weather forecasts, and competitor strategies as observations.

5. **Multi-lap planning:** Current agent decides one lap at a time; a planning horizon across multiple future laps could improve strategic depth.
6. **Better reward shaping:** More intermediate reward signals to guide learning faster.